

Spring Security Guide. Part 2

JWT vs Token opaque

Signature : deux méthodes possibles

On supprime (part-1) AuthUserCache

On crée une classe JwtService

1. createToken(AuthUser)
2. verifyToken(token)

Pourquoi on peut faire confiance au token ?

Quelle librairie utiliser ?

2. Définir la clé secrète dans application.yaml
3. Plus besoin de méthode logout
4. Adapter la méthode login
5. Adapter le filtre de sécurité

JWT vs Token opaque

Token Opaque	JWT (JSON Web Token)
Juste une chaîne aléatoire (UUID)	Contient des infos utiles (encodées)
Stocké côté serveur (en mémoire, Redis...)	Stocké et vérifié directement par le token lui-même
Aucune info visible dedans	Contient un header , un payload et une signature
Nécessite une base de données ou cache pour valider	Peut être validé sans accéder à la base ou au cache

Il contient 3 parties, séparées par des points :

1. **Header** : infos sur le type de token et la méthode de signature (encodé en base64)
2. **Payload** : les **données** (userId, rôles, etc.) (aussi encodé en base64)
3. **Signature** : pour **vérifier** que le token n'a pas été modifié

Signature : deux méthodes possibles

1. Symétrique (HMAC)

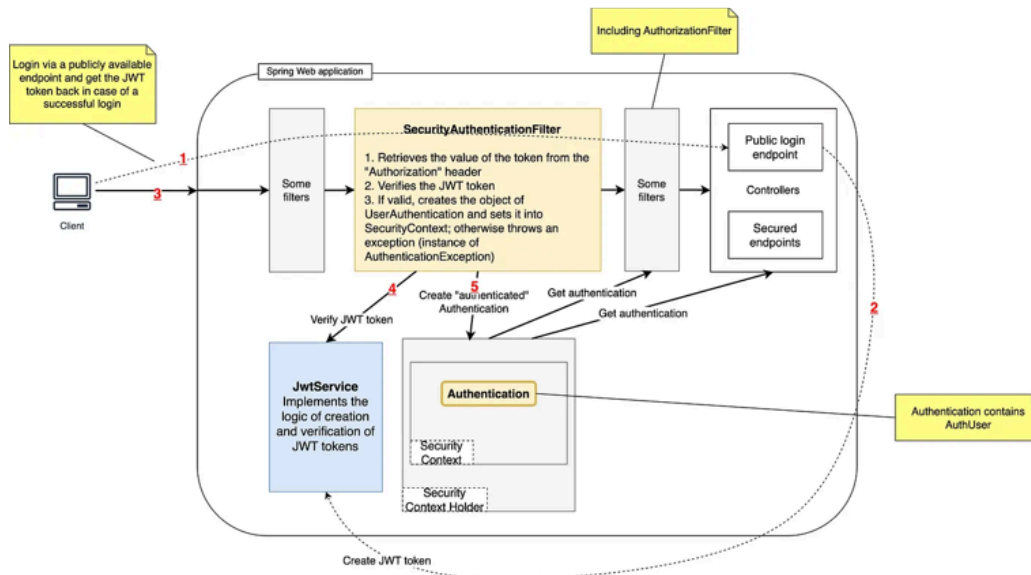
- 1 seule **clé secrète partagée** pour signer **et** vérifier le token.
- Plus simple, mais moins sécurisé si la clé fuit.

2. Asymétrique (RSA / EC)

→ 2 clés : une **privée** (pour signer) et une **publique** (pour vérifier).

→ Plus sécurisé, utilisé dans les gros systèmes.

Dans l'exemple, on utilise **symétrique** (plus simple).



On supprime (part-1) AuthUserCache

Pourquoi ?

Parce que :

- Avec **JWT**, les données de l'utilisateur (id, rôles...) sont **déjà dans le token**.
- On **n'a plus besoin de stocker** les tokens côté serveur.
- Le système devient **stateless** (sans état côté backend).

On crée une classe JwtService

Cette classe aura 2 responsabilités :

```
1. createToken(AuthUser)
```

→ Génère un JWT signé (contenant les infos de l'utilisateur).

```
2. verifyToken(token)
```

→ Vérifie la **signature** du token.

→ Si OK, **extraire les infos** (id, rôles...) pour construire un `AuthUser`.

Pourquoi on peut faire confiance au token ?

- Le token est **signé avec une clé secrète**.
- Si quelqu'un **modifie** le contenu → la **signature ne correspond plus** → rejeté.
- Donc on peut **se fier aux infos** dans le token **sans les vérifier dans la base**.

Quelle librairie utiliser ?

Dans l'exemple :

```
1 <dependency>
2   <groupId>com.auth0</groupId>
3   <artifactId>java-jwt</artifactId>
4   <version>4.0.0</version>
5 </dependency>

1 @Service
2 public class JwtService {
3
4
5   private static final String ROLES_CLAIM = "roles";
6
7
8   private final Algorithm signingAlgorithm;
9
10
11   public JwtService(@Value("${jwt.signing-secret}") String
12   signingSecret) {
13
14     // this example uses a symmetric signature of the JWT token, but if
15     // you want the issuer and the
16     // verifier of the JWT token to be different applications you may
17     // want to use an asymmetric
18     // signature
19
20     this.signingAlgorithm = Algorithm.HMAC256(signingSecret);
21   }
22
23   public AuthUser resolveJwtToken(String token) {
24     try {
25       JWTVerifier verifier = JWT.require(signingAlgorithm).build();
26       DecodedJWT decodedJWT = verifier.verify(token);
27
28       String userId = decodedJWT.getSubject();
29       List<Role> roles =
30       decodedJWT.getClaim(ROLES_CLAIM).asList(Role.class);
31
32
33       return new AuthUser(userId, roles);
34     } catch (JWTVerificationException exception) {
35       throw new TokenAuthenticationException("JWT is not valid");
36     }
37   }
38
39
40   public String createJwtToken(AuthUser authUser) {
41     long nowMillis = System.currentTimeMillis();
42     Date now = new Date(nowMillis);
```

```

43     long expMillis = nowMillis + 3600000; // 1 hour validity
44     Date exp = new Date(expMillis);
45
46
47     List<String> roles =
authUser.roles().stream().map(Role::name).toList();
48
49
50     return JWT.create()
51         .withSubject(authUser.userId())
52         .withClaim(ROLES_CLAIM, roles)
53         .withIssuedAt(now)
54         .withExpiresAt(exp)
55         .sign(signingAlgorithm);
56 }
57 }

```

2. Définir la clé secrète dans application.yaml

```

1 | jwt:
2 |   signing-secret: some-secret

```

En production :

- Ne jamais mettre la clé en dur dans le code ou dans le YAML !
- Utilise une **variable d'environnement** ou un **config server** sécurisé.

3. Plus besoin de méthode logout

- Avec JWT : **"se déconnecter" = supprimer le token côté client** (navigateur, mobile, etc.).
- Si tu veux forcer la déconnexion (**invalider un token avant sa fin de validité**), tu dois :
 - Créer une **blacklist** (liste noire de tokens).
 - Ce n'est **pas fait ici**, car ce tuto reste simple.

4. Adapter la méthode login

Tu ne stockes plus le token, tu **le crées avec** JwtService :

```

1 | @Service
2 | public class AuthService {
3 |
4 |
5 |     private final UserService userService;
6 |
7 |
8 |     private final PasswordEncoder passwordEncoder;
9 |
10 |
11 |     private final JwtService jwtService;
12 |
13 |
14 |     public AuthService(
15 |         UserService userService, PasswordEncoder passwordEncoder,
JwtService jwtService) {
16 |         this.userService = userService;
17 |         this.passwordEncoder = passwordEncoder;
18 |         this.jwtService = jwtService;

```

```

19 }
20
21
22 public TokenDto login(LoginDto loginDto) {
23
24     UserResponseWithCredentials userCredentials =
25         userService.getUserCredentialsByUsername(loginDto.username());
26
27     if (!passwordEncoder.matches(loginDto.password(),
28         userCredentials.passwordHash())) {
29         throw new ApplicationAuthenticationException("Password is
30 incorrect");
31     }
32
33     UserResponse userResponse = userCredentials.userResponse();
34     AuthUser authUser = new AuthUser(userResponse.id(),
35         userResponse.roles());
36
37     String jwtToken = jwtService.createJwtToken(authUser);
38
39     return new TokenDto(jwtToken);
40 }
41 }
42 }

```

5. Adapter le filtre de sécurité

Avant, tu allais chercher l'utilisateur via `AuthUserCache`.

Maintenant, tu **valides et décodes** le token JWT via `JwtService` :

```

1 @Component
2 public class SecurityAuthenticationFilter extends OncePerRequestFilter
3 {
4     private final JwtService jwtService;
5
6     public SecurityAuthenticationFilter(JwtService jwtService) {
7         this.jwtService = jwtService;
8     }
9
10    @Override
11    protected void doFilterInternal(
12        HttpServletRequest request, HttpServletResponse response,
13        FilterChain filterChain)
14        throws ServletException, IOException {
15        String authenticationHeader =
16            request.getHeader(AuthConstants.AUTHORIZATION_HEADER);
17
18        if (authenticationHeader == null) {
19            // Authentication token is not present, let's rely on anonymous
20            authentication
21            filterChain.doFilter(request, response);
22            return;
23        }
24
25        String jwtToken = stripBearerPrefix(authenticationHeader);
26        AuthUser authUser = jwtService.resolveJwtToken(jwtToken);
27    }
28 }

```

```

26     UserAuthentication authentication = new
    UserAuthentication(authUser);
27
28     SecurityContext securityContext =
    SecurityContextHolder.createEmptyContext();
29     securityContext.setAuthentication(authentication);
30     SecurityContextHolder.setContext(securityContext);
31
32     filterChain.doFilter(request, response);
33 }
34
35 String stripBearerPrefix(String token) {
36     if (!token.startsWith("Bearer")) {
37         throw new TokenAuthenticationException("Unsupported
    authentication scheme");
38     }
39 }
40
41 return token.substring(7);
42 }
43 }

```

Changement	Avant (Opaque Token)	Maintenant (JWT)
Stockage token	AuthUserCache (Map ou Redis)	Aucun stockage
Authentification	Lire le token + chercher dans la cache	Lire et valider le token signé
Déconnexion	Supprimer de la cache	Juste oublier le token côté client
Sécurité	Pas de sens dans le token	Données encodées + signées
Clé secrète	Pas utilisée	Requise pour signer/valider le JWT